SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Q.No	Question	Bloom's Level
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze

---

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Heap File Organization

Aim

To write a Python program that simulates a **Heap File Organization** supporting:

1. Insert
2. Delete
3. Sequential scan

Concept

A **heap file** stores records without maintaining any particular ordering based on the key. New records can be inserted into any available location, generally at the end of the file. The main advantage is that insertion is simple and efficient. However, searching for a particular record requires a **sequential scan**, resulting in O(n) search time.

Algorithm

Insertion

1. Accept a record.
2. Append it to the heap file.
3. Assign a unique record ID.

Deletion

1. Search sequentially for the required record.
2. Mark it as deleted or remove it.
3. Confirm deletion.

Sequential Scan

1. Start from the first record.
2. Visit every record.
3. Display all active records.

Python Program

```
class HeapFile:
    def __init__(self):
        self.records = []
        self.next_id = 1

    def insert(self, name, age, department):
        record = {
            "id": self.next_id,
            "name": name,
            "age": age,
            "department": department,
            "active": True
        }

        self.records.append(record)
        self.next_id += 1

        print("Record inserted successfully.")

    def delete(self, record_id):
        for record in self.records:
            if record["id"] == record_id and record["active"]:
                record["active"] = False
                print("Record deleted successfully.")
                return


        print("Record not found.")

    def sequential_scan(self):
        print("\n--- Sequential Scan ---")

        found = False

        for record in self.records:
            if record["active"]:
                print(
                    "ID:", record["id"],
                    "| Name:", record["name"],
                    "| Age:", record["age"],
                    "| Department:", record["department"]
                )
                found = True

        if not found:
            print("Heap file is empty.")


heap = HeapFile()
```

```
heap.insert("Arun", 20, "CSE")
heap.insert("Bala", 21, "ECE")
heap.insert("Kumar", 20, "IT")

heap.sequential_scan()

print("\nDeleting record with ID 2...")
heap.delete(2)

heap.sequential_scan()
```

Output:

```
Record inserted successfully.
Record inserted successfully.
Record inserted successfully.

--- Sequential Scan ---
ID: 1 | Name: Arun | Age: 20 | Department: CSE
ID: 2 | Name: Bala | Age: 21 | Department: ECE
ID: 3 | Name: Kumar | Age: 20 | Department: IT

Deleting record with ID 2...
Record deleted successfully.

--- Sequential Scan ---
ID: 1 | Name: Arun | Age: 20 | Department: CSE
ID: 3 | Name: Kumar | Age: 20 | Department: IT
```

Complexity

Operation	Complexity
Insert	O(1)
Delete	O(n)
Sequential Scan	O(n)
Space	O(n)

Conclusion

Thus, a heap file organization was successfully simulated with **insert, delete, and sequential scan operations**. Heap files provide efficient insertion but require sequential searching for retrieval.

2. Static Hashing with Chaining

Aim

To implement a **static hashing function** for a student database containing 500 records and resolve collisions using **chaining**.

Concept

Static hashing uses a fixed number of buckets. A hash function maps a student's key to a bucket.

For example:

$$h(key)=keymodm$$

where m is the number of buckets.

When two or more keys map to the same bucket, a **collision** occurs. In chaining, multiple records are stored in a linked list or list associated with that bucket.

Python Program

```
class StaticHashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return key % self.size

    def insert(self, roll_no, name, department):
        index = self.hash_function(roll_no)

        record = (roll_no, name, department)

        self.table[index].append(record)

    def search(self, roll_no):
        index = self.hash_function(roll_no)
```

```
        for record in self.table[index]:
            if record[0] == roll_no:
                return record

        return None

    def display(self):
        print("\n--- Hash Table ---")

        for i in range(self.size):
            print("Bucket", i, ":", self.table[i])


# Static hash table
hash_table = StaticHashTable(10)

# Insert student records
for i in range(1, 501):
    hash_table.insert(
        i,
        "Student" + str(i),
        "CSE"
    )

hash_table.display()
```

```
# Search
roll_no = 125
result = hash_table.search(roll_no)

if result:
    print("\nStudent Found:", result)
else:
    print("\nStudent Not Found.")
```

Example

For table size 10:

Roll No = 25

h(25) = 25 % 10

= 5

Therefore, roll number 25 is stored in bucket 5.

Similarly:

15 % 10 = 5

25 % 10 = 5

35 % 10 = 5

These records create a collision and are stored using chaining.

Complexity

Average-case search:

Worst-case search:

Insertion is approximately:

Conclusion

The static hashing system successfully stores **500 student records**. Collisions are handled using **chaining**, allowing multiple records to occupy the same hash bucket.

3. Sorted File Organization with Binary Search

Aim

To simulate a **sorted file organization** and implement **binary search** for retrieving records using the primary key.

Concept

- In sorted file organization, records are maintained in ascending or descending order according to a primary key.
- Because the records are sorted, **binary search** can be used.
- Binary search repeatedly divides the search range into two halves.

Algorithm

1. Store records using the primary key.
2. Sort records by primary key.
3. Set low = 0.
4. Set high = n - 1.
5. Calculate middle position.

6. Compare the middle key with the search key.
7. If equal, return the record.
8. If search key is smaller, search left half.
9. Otherwise, search right half.
10. Continue until found or range becomes empty.

Python Program

```
class SortedFile:
    def __init__(self):
        self.records = []

    def insert(self, roll_no, name, department):
        record = (roll_no, name, department)
        self.records.append(record)

        self.records.sort(key=lambda x: x[0])

    def binary_search(self, key):
        low = 0
        high = len(self.records) - 1

        while low <= high:
            mid = (low + high) // 2

            if self.records[mid][0] == key:
                return self.records[mid]

            elif self.records[mid][0] < key:
                low = mid + 1

            else:
                high = mid - 1
```

```
    def display(self):
        print("\n--- Sorted File ---")

        for record in self.records:
            print(record)


file = SortedFile()

file.insert(105, "Arun", "CSE")
file.insert(101, "Bala", "ECE")
file.insert(110, "Kumar", "IT")
file.insert(103, "Ravi", "CSE")
file.insert(108, "Vijay", "EEE")

file.display()

key = 108

result = file.binary_search(key)

if result:
    print("\nRecord Found:", result)
else:
    print("\nRecord Not Found.")
```

Output:

```
--- Sorted File ---
(101, 'Bala', 'ECE')
(103, 'Ravi', 'CSE')
(105, 'Arun', 'CSE')
(108, 'Vijay', 'EEE')
(110, 'Kumar', 'IT')

Record Found: (108, 'Vijay', 'EEE')
```

Complexity

Operation	Complexity
Binary Search	O(log n)
Display	O(n)
Storage	O(n)

Conclusion

The sorted file organization successfully maintains records according to the primary key and uses **binary search** for efficient retrieval. Binary search significantly improves retrieval compared with sequential search.

4. B-Tree of Order 4

Aim

To implement a **B-Tree of order 4** supporting:

- Insertion
- Searching
- Display

The required keys are:

10, 20, 5, 6, 12, 30, 7, 17

Concept

A B-Tree is a balanced multiway search tree commonly used in database systems and secondary storage.

For a B-Tree of order 4:

- A node can have at most 4 children.
- A node can contain at most 3 keys.
- Keys within a node are sorted.
- All leaf nodes occur at the same level.

Python Program

```
class BTreeNode:
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf


class BTree:
    def __init__(self, order=4):
        self.root = BTreeNode(True)
        self.max_keys = order - 1

    def split_child(self, parent, index):
        child = parent.children[index]

        new_node = BTreeNode(child.leaf)

        middle = child.keys[1]

        new_node.keys = child.keys[2:]
        child.keys = child.keys[:1]

        if not child.leaf:
            new_node.children = child.children[2:]
            child.children = child.children[:2]

        parent.children.insert(index + 1, new_node)
        parent.keys.insert(index, middle)

    def insert(self, key):
        root = self.root

        if len(root.keys) == self.max_keys:
            new_root = BTreeNode(False)
            new_root.children.append(root)

            self.root = new_root

            self.split_child(new_root, 0)
            self.insert_non_full(new_root, key)
```

```
        else:
            self.insert_non_full(root, key)

    def insert_non_full(self, node, key):
        i = len(node.keys) - 1

        if node.leaf:
            node.keys.append(0)

            while i >= 0 and key < node.keys[i]:
                node.keys[i + 1] = node.keys[i]
                i -= 1

            node.keys[i + 1] = key

        else:
            while i >= 0 and key < node.keys[i]:
                i -= 1

            i += 1

            if len(node.children[i].keys) == self.max_keys:
                self.split_child(node, i)

                if key > node.keys[i]:
                    i += 1
            self.insert_non_full(node.children[i], key)

    def search(self, node, key):
        i = 0

        while i < len(node.keys) and key > node.keys[i]:
            i += 1

        if i < len(node.keys) and key == node.keys[i]:
            return True

        if node.leaf:
            return False
```

```
        return self.search(node.children[i], key)

    def display(self, node, level=0):
        print("Level", level, ":", node.keys)

        if not node.leaf:
            for child in node.children:
                self.display(child, level + 1)


btree = BTree()

keys = [10, 20, 5, 6, 12, 30, 7, 17]

for key in keys:
    btree.insert(key)

print("--- B-Tree ---")
btree.display(btree.root)

search_key = 17

if btree.search(btree.root, search_key):
    print("\nKey", search_key, "found.")
else:
    print("\nKey", search_key, "not found.")
```

Output:

```
--- B-Tree ---
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Key 17 found.
```

Complexity

Operation	Complexity
Search	O(log n)
Insert	O(log n)
Space	O(n)

Conclusion

The B-Tree of order 4 was successfully implemented with insertion, searching, and display operations. The tree remains balanced after every insertion.

5. B+ Tree with Leaf-Level Linking

Aim

To implement a **B+ Tree** and demonstrate **leaf-level linking** for efficient range queries on integer keys.

Concept

A B+ Tree differs from a B-Tree because:

- Actual records are stored in leaf nodes.
- Internal nodes contain separator keys.
- Leaf nodes are connected using pointers.
- The linked leaves make range queries highly efficient.

For example:

[5, 10] -> [15, 20] -> [25, 30] -> [35, 40]

After locating the first required key, the linked leaf nodes can be traversed sequentially.

Python Program

```
class BPlusLeaf:
    def __init__(self):
        self.keys = []
        self.next = None


class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.leaves = []

    def insert(self, key):
        leaf = None

        for current in self.leaves:
            if not current.keys or key <= current.keys[-1]:
                leaf = current
                break

        if leaf is None:
            leaf = BPlusLeaf()
            self.leaves.append(leaf)

        leaf.keys.append(key)
        leaf.keys.sort()
```

```
        if len(leaf.keys) > self.order - 1:
            new_leaf = BPlusLeaf()

            mid = len(leaf.keys) // 2

            new_leaf.keys = leaf.keys[mid:]
            leaf.keys = leaf.keys[:mid]

            index = self.leaves.index(leaf)

            self.leaves.insert(index + 1, new_leaf)
```

```
    def link_leaves(self):
        for i in range(len(self.leaves) - 1):
            self.leaves[i].next = self.leaves[i + 1]

    def display(self):
        print("\n--- B+ Tree Leaves ---")

        for i, leaf in enumerate(self.leaves):
            print("Leaf", i, ":", leaf.keys)

    def range_query(self, start, end):
        result = []

        for leaf in self.leaves:
            for key in leaf.keys:
                if start <= key <= end:
                    result.append(key)

        return result


tree = BPlusTree()

keys = [5, 10, 15, 20, 25, 30, 35, 40, 45]

for key in keys:
    tree.insert(key)

tree.link_leaves()

tree.display()

print("\nRange Query (15 to 35):")
print(tree.range_query(15, 35))
```

Output:

```
--- B+ Tree Leaves ---
Leaf 0 : [5]
Leaf 1 : [10]
Leaf 2 : [15]
Leaf 3 : [20]
Leaf 4 : [25]
Leaf 5 : [30]
Leaf 6 : [35]
Leaf 7 : [40]
Leaf 8 : [45]

Range Query (15 to 35):
[15, 20, 25, 30, 35]
```

Advantage

The leaf-level links eliminate the need to repeatedly traverse the internal tree when retrieving a range of values.

Complexity

Point search:

O(logn)

Range query:

O(logn+k)

where k is the number of matching records.

Conclusion

The B+ Tree was implemented with linked leaf nodes. The leaf-level linking makes sequential range retrieval efficient and is one of the major reasons B+ Trees are widely used in database indexing.

6. Dense Primary Index on a Sorted File

Aim

To implement a **dense primary index** on a sorted file and support:

1. Search using index key
2. Display index table

Concept

A **dense index** contains an index entry for **every record** in the data file.

Example:

Index Key	Address
101	0
102	1
103	2
104	3

Because every record has an index entry, direct searching through the index is efficient.

Python Program

```
class DenseIndex:
    def __init__(self):
        self.records = []
        self.index = []

    def add_record(self, key, name):
        self.records.append((key, name))

    def build_index(self):
        self.records.sort(key=lambda x: x[0])

        self.index = []

        for position, record in enumerate(self.records):
            key = record[0]
            self.index.append((key, position))
```

```
    def display_index(self):
        print("\n--- Dense Primary Index ---")
        print("Key\tAddress")

        for key, address in self.index:
            print(key, "\t", address)

    def search(self, key):
        low = 0
        high = len(self.index) - 1

        while low <= high:
            mid = (low + high) // 2

            if self.index[mid][0] == key:
                address = self.index[mid][1]
                return self.records[address]

            elif self.index[mid][0] < key:
                low = mid + 1

            else:
                high = mid - 1

        return None
index = DenseIndex()

index.add_record(104, "Arun")
index.add_record(101, "Bala")
index.add_record(105, "Kumar")
index.add_record(102, "Ravi")
index.add_record(103, "Vijay")

index.build_index()

index.display_index()

key = 103
result = index.search(key)

if result:
    print("\nRecord Found:", result)
else:
    print("\nRecord Not Found.")
```

Output:

```
--- Dense Primary Index ---
Key        Address
101         0
102         1
103         2
104         3
105         4

Record Found: (103, 'Vijay')
```

Complexity

Index search:

O(logn)

Index construction:

O(nlogn)

Space:

O(n)

Conclusion

The dense primary index successfully creates one index entry for every record. Binary search on the sorted index allows efficient retrieval.

7. Extendible Hashing

Aim

To implement **extendible hashing** and demonstrate **directory doubling** when overflow occurs during insertion of 8 records.

Concept

Extendible hashing is a **dynamic hashing technique**.

It uses:

- Directory
- Global depth
- Local depth
- Hash buckets

When a bucket overflows:

1. If local depth is less than global depth, split the bucket.
2. If local depth equals global depth, double the directory.
3. Increase global depth.
4. Redistribute the records.

Python Program

```
class Bucket:
    def __init__(self, depth, capacity=2):
        self.local_depth = depth
        self.capacity = capacity
        self.records = []


class ExtendibleHash:
    def __init__(self, bucket_capacity=2):
        self.global_depth = 1
        self.bucket_capacity = bucket_capacity

        bucket0 = Bucket(1, bucket_capacity)
        bucket1 = Bucket(1, bucket_capacity)

        self.directory = [bucket0, bucket1]

    def hash_function(self, key):
        return key & ((1 << self.global_depth) - 1)

    def double_directory(self):
        old_directory = self.directory

        self.directory = old_directory + old_directory
        self.global_depth += 1
```

```
        print("\nDirectory doubled.")
        print("Global depth:", self.global_depth)

    def split_bucket(self, index):
        old_bucket = self.directory[index]

        old_depth = old_bucket.local_depth

        old_bucket.local_depth += 1

        new_bucket = Bucket(
            old_bucket.local_depth,
            self.bucket_capacity
        )

        pattern = 1 << (old_bucket.local_depth - 1)

        for i in range(len(self.directory)):
            if self.directory[i] is old_bucket:
                if i & pattern:
                    self.directory[i] = new_bucket

        old_records = old_bucket.records
        old_bucket.records = []

        for key, value in old_records:
            new_index = self.hash_function(key)
            self.directory[new_index].records.append(
                (key, value)
            )

    def insert(self, key, value):
        while True:
            index = self.hash_function(key)
            bucket = self.directory[index]

            if len(bucket.records) < bucket.capacity:
                bucket.records.append((key, value))
                return
```

```
            if bucket.local_depth == self.global_depth:
                self.double_directory()

            self.split_bucket(index)

    def display(self):
        print("\n--- Extendible Hashing ---")

        for i, bucket in enumerate(self.directory):
            print(
                "Directory", i,
                "| Local Depth:", bucket.local_depth,
                "| Records:", bucket.records
            )


eh = ExtendibleHash()

records = [
    (1, "A"),
    (2, "B"),
    (3, "C"),
    (4, "D"),
    (5, "E"),
    (6, "F"),
    (7, "G"),
    (8, "H")
]

for key, value in records:
    eh.insert(key, value)

eh.display()
```

Output:

```
Directory doubled.
Global depth: 2

--- Extendible Hashing ---
Directory 0 | Local Depth: 2 | Records: [(4, 'D'), (8, 'H')]
Directory 1 | Local Depth: 2 | Records: [(1, 'A'), (5, 'E')]
Directory 2 | Local Depth: 2 | Records: [(2, 'B'), (6, 'F')]
Directory 3 | Local Depth: 2 | Records: [(3, 'C'), (7, 'G')]
```

Important Observation

Initially:

Global Depth = 1

Directory Size = 2

When a bucket becomes full and cannot be split using the current global depth:

Global Depth increases

Directory is doubled

For example:

2 entries

↓

Overflow

↓

Directory doubling

↓

Bucket splitting

↓

Records redistributed

Complexity

Average insertion:

O(1)

Average search:

O(1)

Directory space:

$O(2^d)$

where d is global depth.

Conclusion

Extendible hashing dynamically expands its directory when overflow occurs. Therefore, it can adapt to increasing data without requiring a complete rehash of the entire database.

8. Secondary Storage Block Access — Sequential vs Indexed Retrieval

Aim

To simulate secondary storage block access and calculate the number of **I/O operations** required for sequential and indexed retrieval.

Concept

In database systems, records are stored in disk blocks. Accessing a disk block generally requires an I/O operation.

Suppose:

Number of records = 100

Records per block = 10

Then:

Number of blocks = 100/10 = 10

Python Program

```
import math


def calculate_blocks(records, records_per_block):
    return math.ceil(records / records_per_block)


def sequential_io(records, records_per_block, target_position):
    blocks = calculate_blocks(records, records_per_block)

    target_block = target_position // records_per_block

    # Sequential search may require reading
    # all blocks up to the target block.
    io_operations = target_block + 1

    return blocks, io_operations


def indexed_io(records, records_per_block):
    data_blocks = calculate_blocks(records, records_per_block)

    # One I/O for index and one I/O for data block
    return 2
```

```
records = 100
records_per_block = 10
target_position = 75

blocks, sequential = sequential_io(
    records,
    records_per_block,
    target_position
)
```

```
indexed = indexed_io(records, records_per_block)

print("Total Records:", records)
print("Records per Block:", records_per_block)
print("Total Data Blocks:", blocks)

print("\nTarget Record Position:", target_position)

print("Sequential Retrieval I/O:",
    sequential)

print("Indexed Retrieval I/O:",
    indexed)
```

Output:

```
Total Records: 100
Records per Block: 10
Total Data Blocks: 10

Target Record Position: 75
Sequential Retrieval I/O: 8
Indexed Retrieval I/O: 2
```

Comparison

Method	I/O Operations
Sequential	Depends on target location
Indexed	Approximately 2 for simple one-level index

Conclusion

Indexed retrieval significantly reduces disk I/O compared with sequential retrieval, especially when the database contains a large number of records.

9. Sparse Index vs Dense Index with Timing Analysis

Aim

To implement a **sparse index** on a sorted file and compare its search performance with a **dense index** using timing analysis.

Concept

- A dense index contains an index entry for every record.
- A sparse index contains index entries only for selected records, commonly one entry per block.

Example

Suppose:

Records:

10 20 30 40 50 60 70 80

With block size 2, a sparse index could be:

10 → Block 0

30 → Block 1

50 → Block 2

70 → Block 3

This consumes less memory than a dense index.

Python Program

```
import time
import bisect


def create_records(n):
    return [(i, "Student" + str(i)) for i in range(n)]


def create_dense_index(records):
    return [(record[0], i)
            for i, record in enumerate(records)]
```

```
def create_sparse_index(records, block_size):
    index = []

    for i in range(0, len(records), block_size):
        index.append((records[i][0], i))

    return index


def dense_search(records, index, key):
    keys = [item[0] for item in index]

    position = bisect.bisect_left(keys, key)

    if position < len(keys) and keys[position] == key:
        return records[index[position][1]]

    return None


def sparse_search(records, index, key, block_size):
    keys = [item[0] for item in index]

    position = bisect.bisect_right(keys, key) - 1

    if position < 0:
        position = 0

    start = index[position][1]
    end = min(start + block_size, len(records))

    for i in range(start, end):
        if records[i][0] == key:
            return records[i]

    return None
```

```
records = create_records(100000)

block_size = 100

dense_index = create_dense_index(records)
sparse_index = create_sparse_index(
    records,
    block_size
)

search_key = 98765

start = time.perf_counter()

result1 = dense_search(
    records,
    dense_index,
    search_key
)

dense_time = time.perf_counter() - start


start = time.perf_counter()

result2 = sparse_search(
    records,
    sparse_index,
    search_key,
    block_size
)

sparse_time = time.perf_counter() - start
```

```
print("Dense Index Result:", result1)
print("Sparse Index Result:", result2)

print("\nDense Index Time:",
    dense_time)

print("Sparse Index Time:",
    sparse_time)

print("\nDense Index Entries:",
    len(dense_index))

print("Sparse Index Entries:",
    len(sparse_index))
```

Output:

```
Dense Index Result: (98765, 'Student98765')
Sparse Index Result: (98765, 'Student98765')

Dense Index Time: 0.0034053999697789 55
Sparse Index Time: 9.779998799785972e-05

Dense Index Entries: 100000
Sparse Index Entries: 1000
```

Comparison

For 100,000 records and block size 100:

Dense index entries = 100000

Sparse index entries = 1000

Therefore, the sparse index requires much less index storage.

However, after locating the correct block, the sparse index may need a short sequential search inside that block.

Comparison Table

Feature	Dense Index	Sparse Index
Index entries	Every record	Selected records
Space	Higher	Lower
Search	Faster	Slightly slower
Suitable for	Frequently searched data	Large sorted files
Storage overhead	High	Low

Conclusion

The timing experiment demonstrates the trade-off between index size and search performance. A dense index generally provides faster direct retrieval, while a sparse index saves considerable memory and storage space.

10. B+ Tree Construction and Range Query

Aim

To construct and traverse a **B+ Tree**, then perform a range query for keys between:

15 and 45

and display all matching records.

Concept

B+ Trees are particularly suitable for range queries because all data records are stored in sorted leaf nodes, and the leaves are connected using pointers.

For:

Range = 15 to 45

the system finds the first leaf containing 15 and then follows the leaf links until reaching a key greater than 45.

Python Program

```
class LeafNode:
    def __init__(self):
        self.keys = []
        self.next = None


class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.leaves = []

    def insert(self, key):
        # If there are no leaves
        if not self.leaves:
            leaf = LeafNode()
            leaf.keys.append(key)
            self.leaves.append(leaf)
            return

        # Find appropriate leaf
        selected = None

        for leaf in self.leaves:
            if key <= leaf.keys[-1]:
                selected = leaf
                break

        if selected is None:
            selected = self.leaves[-1]

        selected.keys.append(key)
        selected.keys.sort()

        # Split overflowing leaf
        if len(selected.keys) > self.order - 1:
            new_leaf = LeafNode()
```

```
        mid = len(selected.keys) // 2

        new_leaf.keys = selected.keys[mid:]
        selected.keys = selected.keys[:mid]

        position = self.leaves.index(selected)

        self.leaves.insert(
            position + 1,
            new_leaf
        )

    def connect_leaves(self):
        for i in range(len(self.leaves) - 1):
            self.leaves[i].next = self.leaves[i + 1]

    def display(self):
        print("\n--- B+ Tree Traversal ---")

        for i, leaf in enumerate(self.leaves):
            print(
                "Leaf", i,
                ":", leaf.keys
            )

    def range_query(self, start, end):
```

```
        # Locate starting leaf
        while current:
            if current.keys and current.keys[-1] >= start:
                break

            current = current.next

        # Traverse linked leaves
        while current:
            for key in current.keys:
                if start <= key <= end:
                    result.append(key)
```

```
            elif key > end:
                return result

            current = current.next

        return result


tree = BPlusTree(order=4)

keys = [
    5, 10, 15, 20, 25,
    30, 35, 40, 45, 50, 55
]

for key in keys:
    tree.insert(key)

tree.connect_leaves()

tree.display()

start = 15
end = 45

result = tree.range_query(start, end)

print("\nRange Query:", start, "to", end)

print("Matching Records:")

for key in result:
    print(key)
```

Output:

```
--- B+ Tree Traversal ---
Leaf 0 : [5, 10]
Leaf 1 : [15, 20]
Leaf 2 : [25, 30]
Leaf 3 : [35, 40]
Leaf 4 : [45, 50, 55]

Range Query: 15 to 45
Matching Records:
15
20
25
30
35
40
45
```

Range Query Process

The search begins with the first leaf containing the lower boundary: **15**

Then the linked leaves are traversed:

15 → 20 → 25 → 30 → 35 → 40 → 45

The traversal stops when the key becomes greater than: **45**

Thus the matching records are:

15, 20, 25, 30, 35, 40, 45

Complexity

Finding the starting point:

O(logn)

Traversing k matching records:

O(k)

Overall:

O(logn+k)

Conclusion

The B+ Tree was successfully constructed and traversed. The linked leaf nodes enabled efficient range retrieval of all records between **15 and 45**.